

Criterion: Observability (Logging + Error Monitoring)

Architecture Decision Record

Status

Status: Accepted

Date: 2026-01-05

Context

The MVP needs enough observability to diagnose API issues and client crashes during demos: request logging, structured app logs, and error reporting. The setup must remain lightweight.

Decision

Use Sentry for error monitoring (frontend + backend). Use Winston for backend application logs and Morgan for HTTP request logs. Add a frontend Error Boundary to prevent blank screens and capture exceptions.

Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Full APM suite	Rich diagnostics	Cost + setup	Too heavy for MVP
Console-only logs	Minimal work	Hard triage	No aggregation/structure
No client monitoring	Simpler	Silent UI failures	Poor UX and no visibility

Consequences

Positive: faster debugging and clearer demo reliability.

Negative: Sentry must be configured to avoid sending sensitive data.

Neutral: sampling/verbosity can be tuned later.

Implementation Details

Key Implementation Decisions

- * Skip noisy endpoints (e.g., health checks) in request logging.
- * Keep monitoring optional via environment variables.

Project Structure

frontend/src/main.tsx
frontend/src/app/providers/ErrorBoundary/
backend/src/config/instrument.js
backend/src/utils/logger.ts
backend/src/utils/requestLogger.ts

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Frontend crash fallback	[OK]	ErrorBoundary
2	Frontend error reporting	[OK]	Sentry init
3	Backend error reporting	[OK]	Sentry init + handlers
4	Backend structured logs	[OK]	Winston logger
5	HTTP request logs	[OK]	Morgan middleware

Known Limitations

Limitation	Impact	Potential Solution
Sentry may capture extra context	Privacy risk	Keep PII/PHI out of the app and disab...
Monitoring disabled if DSN missing	Less visibility	Require DSN for staging/prod demos